The Tech Shop Challenge
Create a Dynamic Webpage using JS

The Tech Shop Challenge

Build a dynamic webpage using "vanilla" JavaScript. Follow the guidelines given below, but feel free to customize the app if you want.

It must meet the minimum requirements prescribed.

---

1) Set up the development environment

Make sure you have installed at least the following tools:

- Visual Studio Code
- Live Server VS extension

You will also use your Browser's Developer Tools for testing and debugging.

2) Review the given wireframe

You can follow the wireframe provided (Appendix B) or you can create your own design if it meets the minimum requirements set out below in Step 4.

3) Write HTML & CSS as needed

Use HTML and CSS to structure and design your application.

---

*You **may** use a CSS library like Bootstrap to style your webpage.*

---

4) Use plain JavaScript (no frameworks) to build the following minimum requirements into your webpage (See Appendix A for detailed specs):

1) **The Bank** – an area where you will store funds and make bank loans
2) **Work** – an area to increase your earnings and deposit cash into your bank balance
3) **Laptops** – an area to select and display information about the merchandise

5) Submit

a) If you created your own wireframe, upload it as PDF to Moodle.
b) Create a git repository and ensure its visibility is set to "private" and add your lecturer as a maintainer. Upload the latest version of your app before the deadline of the assignment.
c) **Additional challenge**: Try to host your web page using a hosting provider of your choice.
   a) Vercel
   b) GitLab Pages (Requires adding of card information but is free)
   c) GitHub Pages

Appendix A: Requirements for the Tech shop

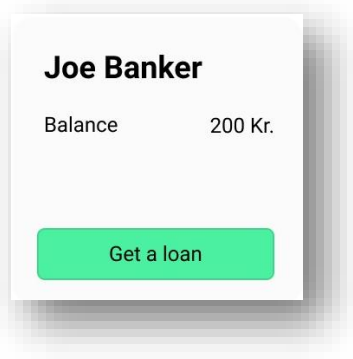


Figure 1 The Bank Section

1. The Bank (Figure 1)

1.1 Balance

The bank shows a "Bank" balance in your currency. This is the amount available for you to buy a laptop.

TIP: Look at the Number format feature in JavaScript

1.2 Outstanding Loan (Only visible after taking a loan)

Shows the outstanding Loan value. This should be reduced as the loan is paid back.

1.3 Get a loan

The Get a loan button will attempt to get a loan from the bank. When the Get a loan button is clicked, it must show a "Prompt" popup box that allows you to enter an amount.

Constraints on Get a loan button:

1. You *cannot* get a loan more than double of your bank balance (*i.e.*, If you have 500 you *cannot* get a loan greater than 1000.)
2. You may not have two loans at once. The initial loan should be paid back in full.

***TIP**: Remember to check for value types before doing mathematical operations*

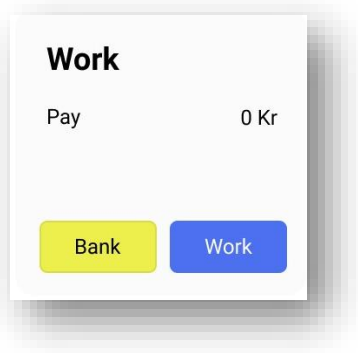


Figure 2 Work Section

2. Work (Figure 2)

2.1 Pay

The pay or your current salary amount in your currency. Should show how much money you have earned by "working". This money is NOT part of your bank balance.

2.2 Bank Button

The bank button must transfer the money from your *Pay/Salary balance* to your *Bank balance.* Remember to reset your pay/salary once you transfer.

Constraints on Bank button:

1. If you have an outstanding loan, 10% of your salary MUST first be deducted and transferred to the outstanding Loan amount.
2. The balance after the 10% deduction may be transferred to your bank account.

2.3 Work button

The work button must increase your *Pay balance* at a rate of 100 on each click.

2.4 Repay Loan button

Once you have a loan, a new button labeled "Repay Loan" should appear. Upon clicking this button, the full value of your current **Pay** amount should go towards the outstanding loan and NOT your bank account.

Any remaining funds after paying the loan may be transferred to your bank account.

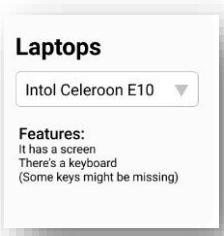


Figure 3 Laptop Selection Area

Figure 4 Laptop Information Area

3. Laptops

The laptops section has 2 parts: laptop selection area (Figure 3) and info section (Figure 4)

3.1 Laptop selection (Figure 3)

Use a select box to show the available computers. The feature list of the selected laptop must be displayed here. Changing a laptop should update the user interface with the information for that selected laptop.

3.1.1 Laptop API

The data for the laptops will be provided to you via a RESTful API that returns JSON data.

The endpoint for the API is: https://screeching-lopsided-canidae.glitch.me/computers

The endpoint will return an array of computers, each computer will have the following properties:

```
{
    "id": 1,
    "title": "Classic Notebook",
    "description": "A little old, but turns on.",
    "specs": [
      "Has a screen",
      "Keyboard works, mostly",
      "32MB Ram (Not upgradable)",
      "6GB Hard Disk",
      "Comes with Floppy Disk Reader (Free) - Requires cable",
      "Good exercise to carry"
    ],
    "price": 200,
    "stock": 1,
    "active": true,
    "image": "1.jpg"
}
```

Use this API to populate your laptop selection dropdown box.

3.2 Info section (Figure 4)

The Info section is where the image, name, and description as well as the price of the laptop must be displayed.

3.2.1 Images

The base URL + the path to the image file must be used to retrieve an image. The path to the image of a laptop can be found in the response as seen above. The base URL can be retrieved from the endpoint: https://screeching-lopsided-canidae.glitch.me/config. If an image doesn't load correctly, make sure you have assigned appropriate *alt* text to be displayed.

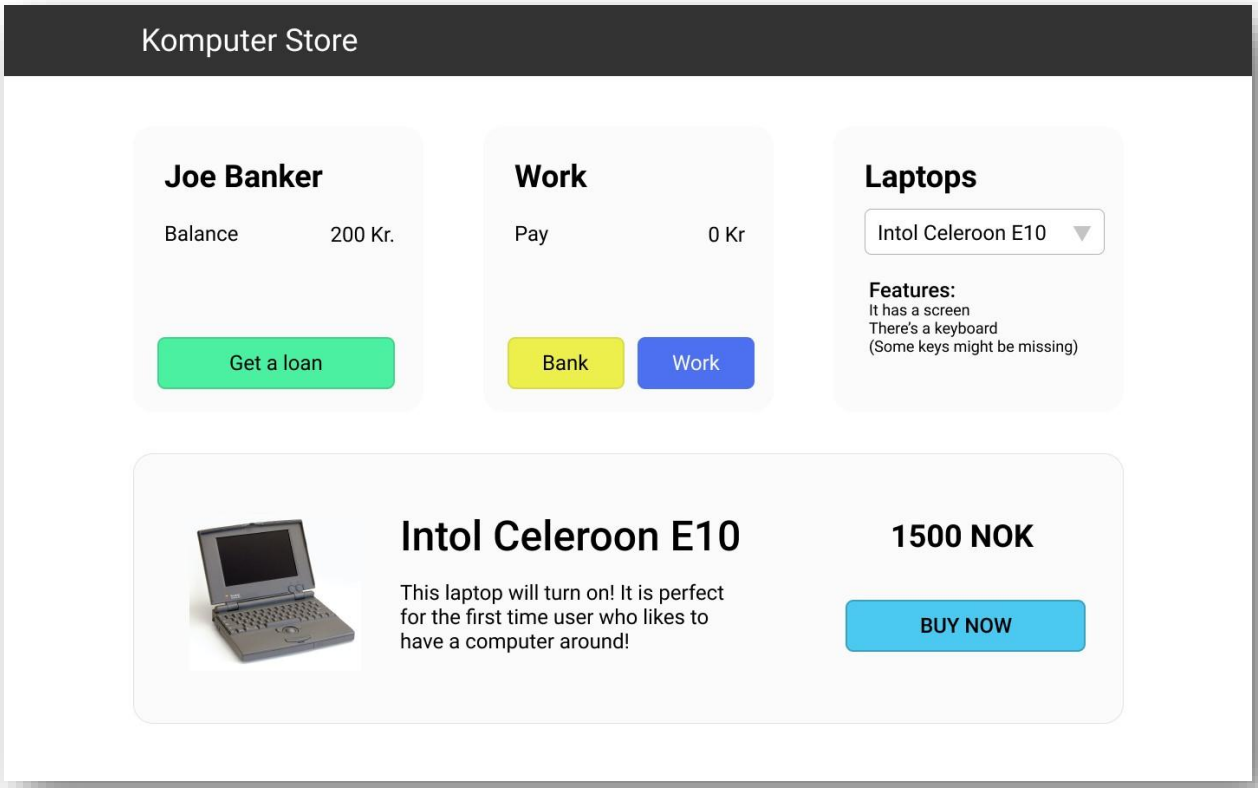
Example Image *src* Link:
https://cdn.glitch.global/638b3c0b-25db-4de8-bb86-38ead3289b23/1.jpg

---

3.2.2 Buy Now button

The buy now button is the final action of your website. This button will attempt to “Buy” a laptop and validate whether the bank balance is sufficient to purchase the selected laptop.

If you do not have enough money in the “Bank”, a message must be shown that you cannot afford the laptop.

When you have sufficient “Money” in the account, the amount must be deducted from the bank, and you must receive a message that you are now the owner of the new laptop!

Appendix B: Sample Wireframe for the Komputer Store